

Manual do Sistema — Consultor de Veículos

Documento: Manual do Sistema — Agente Consultor de Veículos **Versão:** 1.0.0 **Data de emissão:** agosto de 2026 **Departamento:** Tecnologia / Base de Conhecimento Corporativa **Classificação:** Interno — uso técnico **Responsável:** Equipe de Tecnologia, Autoluz Veículos

Tabela de conteúdos

- 1 Introdução e propósito
 - 2 Visão geral do sistema
 - 3 Princípios de projeto
 - 4 Arquitetura
 - 5 Catálogo de componentes
 - 6 Catálogo de ferramentas do agente
 - 7 Ciclo de vida de uma pergunta
 - 8 Pipeline de dados em tempo de construção
 - 9 Base de conhecimento e formatos aceitos
 - 10 Camada de recuperação semântica
 - 11 Regras de comportamento e recusa
 - 12 Configuração
 - 13 Implantação
 - 14 Observabilidade e registro de execução
 - 15 Segurança e privacidade
 - 16 Testes e qualidade
 - 17 Manutenção e operação contínua
 - 18 Limitações conhecidas
 - 19 Registros de decisão de arquitetura
 - 20 Glossário
 - 21 Processo de atualização deste documento
-

Seção 1 — Introdução e propósito

1.1 Propósito do documento

Este documento é a referência técnica canônica do **Consultor de Veículos**, agente de inteligência artificial que atende os colaboradores da Autoluz Veículos. Destina-se a quem precisa operar, manter, auditar ou evoluir o sistema: equipe de tecnologia, responsáveis pelas áreas de negócio que mantêm os documentos indexados e qualquer pessoa encarregada de avaliar a confiabilidade das respostas.

Ele descreve **o que o sistema faz, como faz e por que foi construído assim**. Não substitui o `README.md` do repositório, voltado a quem vai executar o projeto, nem as políticas internas da empresa, que são o conteúdo que o agente consulta e não a descrição do sistema.

1.2 Contexto

A Autoluz Veículos é uma rede de concessionárias multimarcas com 14 unidades. A informação de que um colaborador precisa no atendimento está espalhada por fontes de natureza distinta: tabela de preços de referência, tabelas de consumo publicadas por órgão regulador, manual do fabricante do veículo e as políticas internas da própria rede.

Antes do agente, responder "o cliente atrasou a revisão, perdeu a garantia?" exigia localizar a política correta, ler a seção pertinente e interpretar. A resposta variava conforme quem atendia — que é exatamente o problema que uma base de conhecimento conversacional resolve.

1.3 O que caracteriza este sistema

Três características o distinguem de um assistente genérico:

É fechado por construção. O agente responde sobre 28 modelos e um conjunto declarado de documentos. Fora disso, ele diz que não sabe. Essa fronteira é explícita no prompt, verificada por testes e visível na interface.

Não faz conta de cabeça. Todo número — preço, consumo, custo de viagem — vem de uma ferramenta determinística. O modelo de linguagem interpreta a pergunta, escolhe a ferramenta e redige a resposta; ele nunca produz um valor por conta própria.

Separa tempo de construção de tempo de execução. Coleta, extração e cálculo de embeddings acontecem antes, em scripts, e o resultado é versionado. Em produção o sistema apenas lê.

Seção 2 — Visão geral do sistema

2.1 Capacidades

Capacidade	Fonte da informação
Ficha técnica de veículo	Catálogo estruturado em CSV
Preço de referência	Tabela FIPE, coletada por script
Consumo e eficiência energética	PBE Veicular do Inmetro
Preço de combustível por estado	Levantamento da ANP
Simulação de custo de viagem	Cálculo determinístico em Python
Políticas internas da empresa	Busca semântica no acervo corporativo
Manual do proprietário	Busca semântica no manual indexado
Metodologia de etiquetagem	Busca semântica nos documentos do Inmetro

2.2 Interfaces

Duas interfaces consomem o mesmo agente, sem duplicar regra de negócio:

- **Web (Streamlit)** — chat com histórico de sessão, exemplos de partida, tutorial de primeiro acesso e avaliação por resposta;
- **Linha de comando** — uso pontual e automação; aceita pergunta única como argumento ou modo interativo.

Uma terceira interface — API, bot de mensageria, widget de intranet — exigiria apenas um arquivo novo em `app/`, chamando `criar_agente()`.

2.3 Público

O agente é aberto a **qualquer colaborador**, sem restrição de acesso por área ou cargo. Não há autenticação: o conteúdo indexado é material de circulação interna irrestrita. Documento de acesso controlado não entra na base — a decisão de o que indexar é o controle de acesso.

Seção 3 — Princípios de projeto

P-01 — Fronteira explícita. O sistema declara o que cobre. Uma pergunta fora do escopo recebe recusa fundamentada, com indicação da área responsável, em vez de resposta plausível e não verificada.

P-02 — Número vem de ferramenta. Modelos de linguagem erram aritmética de forma silenciosa e confiante. Todo cálculo é função Python coberta por teste.

P-03 — Recuperação semântica só para texto corrido. Busca vetorial é boa para "o que diz a política sobre X" e ruim para "o mais econômico até 100 mil". Consulta estruturada resolve o segundo caso, com pandas.

P-04 — Toda resposta cita a fonte. Documento e página, quando houver. Sem rastreabilidade, a resposta não é auditável.

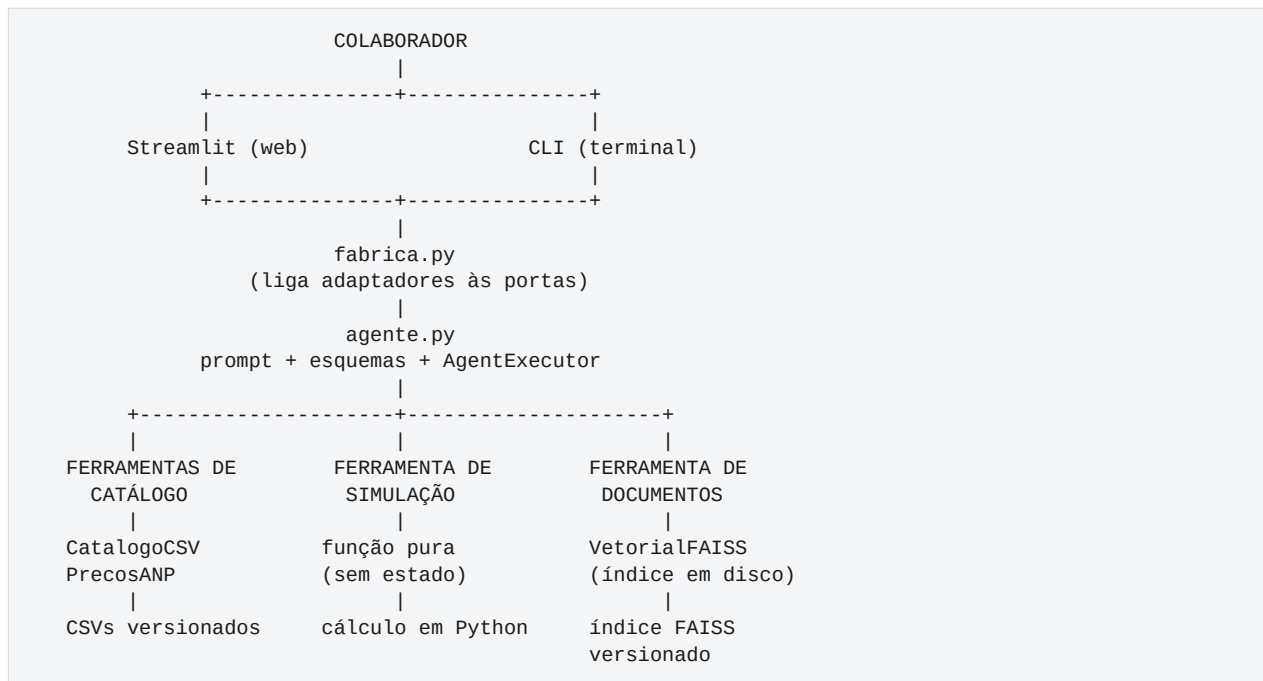
P-05 — Dependência externa fora do caminho crítico. Nenhuma API de terceiro é chamada durante a conversa, exceto o provedor de linguagem.

P-06 — Troca de tecnologia é troca de um arquivo. Provedor de IA, base vetorial e fonte de dados são contratos, não implementações.

P-07 — Falha de observabilidade não derruba função. O registro de execução falha em silêncio; a conversa continua.

Seção 4 — Arquitetura

4.1 Diagrama textual



4.2 Camadas

O projeto segue **arquitetura hexagonal**, com quatro camadas de dependência unidirecional — a de dentro nunca conhece a de fora.

Camada	Pacote	Conhece
Domínio	dominio/	Nada além da biblioteca padrão
Ferramentas	ferramentas/	Domínio
Adaptadores	adaptadores/	Domínio e bibliotecas externas
Composição	fabrica.py, agente.py	Tudo
Interface	app/	Apenas criar_agente() e responder()

O domínio não importa LangChain, Streamlit, FAISS, pandas ou qualquer provedor. Isso é verificável: dominio/modelos.py e dominio/portas.py importam apenas dataclasses, enum e typing.

4.3 Portas

Quatro contratos, declarados como Protocol do Python — qualquer classe com os métodos corretos satisfaz, sem herança:

Porta	Métodos	Implementação atual
ProvedorLLM	modelo_chat, modelo_embedding	ProvedorGemini, ProvedorNVIDIA
BaseVetorial	buscar	VetorialFAISS
RepositorioCatalogo	listar, buscar_por_nome, filtrar	CatalogoCSV
RepositorioPrecosCombustivel	preco, por_estado, estados_disponiveis	PrecosANP

4.4 Custo real de uma troca

A intercambialidade não é teórica. Durante a construção do projeto, o acesso à API da NVIDIA foi bloqueado na verificação de conta. A migração para o Google Gemini custou **um arquivo novo** (`adaptadores/llm_gemini.py`) e **uma linha** no dicionário `PROVEDORES` da fábrica. Agente, ferramentas, dados, testes e interface não foram tocados. O histórico de commits registra a mudança inteira.

Seção 5 — Catálogo de componentes

5.1 Domínio

Arquivo	Responsabilidade
<code>dominio/modelos.py</code>	<code>Veiculo</code> , <code>TrechoRecuperado</code> , <code>PrecoCombustivel</code> , <code>ResultadoViagem</code> , <code>CustoPorCombustivel</code> , <code>enum Combustivel</code>
<code>dominio/portas.py</code>	Os quatro contratos da seção 4.3

5.2 Adaptadores

Arquivo	Responsabilidade
<code>adaptadores/llm_gemini.py</code>	Modelo de chat e de embeddings do Google Gemini
<code>adaptadores/llm_nvidia.py</code>	Mesmo contrato, sobre NVIDIA NIM
<code>adaptadores/vetorial_faiss.py</code>	Carrega o índice FAISS do disco; busca com filtro de metadado e corte por relevância
<code>adaptadores/catalogo_csv.py</code>	Consulta, filtro e ordenação sobre os CSVs, com pandas
<code>adaptadores/precos_anp_csv.py</code>	Preço de combustível por unidade da federação

5.3 Ferramentas

Arquivo	Responsabilidade
<code>ferramentas/consultar_catalogo.py</code>	Busca, listagem, comparação e resumo do catálogo
<code>ferramentas/simular_viagem.py</code>	Cálculo de litros, custo total e custo por quilômetro
<code>ferramentas/consultar_precos.py</code>	Preço por estado e ranking entre estados
<code>ferramentas/buscar_documentos.py</code>	Busca semântica com filtro e limiar
<code>ferramentas/formato.py</code>	Formatação de números, moeda e datas em português

5.4 Composição e apoio

Arquivo	Responsabilidade
<code>config.py</code>	Ponto único de leitura do ambiente e de caminhos
<code>agente.py</code>	Prompt, esquemas das ferramentas, executor e <code>responder()</code>
<code>fabrica.py</code>	Instancia adaptadores e monta o agente

Arquivo	Responsabilidade
registro.py	Registro de execução e de feedback, em JSON Lines
documentos.py	Extração de texto por formato de arquivo

5.5 Regra de configuração

Nenhum módulo além de `config.py` lê `os.environ`. A regra é verificável por inspeção e existe para que trocar provedor, modelo ou local de dados seja alteração em um único ponto.

Seção 6 — Catálogo de ferramentas do agente

O modelo recebe oito ferramentas com esquema de argumentos tipado. A descrição de cada uma é o que determina se ele a escolhe corretamente — descrição vaga produz ferramenta errada, e o sintoma aparece como resposta ruim.

Ferramenta	Argumentos	Devolve
resumo_catalogo	nenhum	Quantidade de veículos, marcas, categorias e faixa de preço
buscar_veiculo	termo	Ficha completa dos veículos cujo nome corresponde
listar_veiculos	marca, categoria, preco_maximo, preco_minimo, combustivel, ordenar_por	Lista filtrada e ordenada
comparar_veiculos	termos (2 a 5)	Fichas lado a lado
simular_viagem	veiculo, distancia_km, percentual_cidade, ida_e_volta, uf, preços opcionais	Litros, custo total, custo por km e tanques necessários
consultar_precos_combustivel	uf	Preço de gasolina, etanol e diesel no estado
ranking_precos_por_estado	produto	Estados ordenados por preço
buscar_documentos_oficiais	consulta, tipo	Trechos com fonte e página

6.1 Notas de projeto por ferramenta

`resumo_catalogo` **existe por um defeito observado**. Perguntado "quais marcas vocês têm?", o modelo respondia a partir de uma listagem truncada e **omitia marcas inteiras** — erro grave, porque a resposta parecia completa. A ferramenta devolve o agregado, e o prompt obriga a usá-la para perguntas de cobertura.

`simular_viagem` **soma litros por trecho**. Aplicar a média dos consumos sobre a distância total **subestima o gasto**, porque consumo é razão invertida: um veículo que faz 8 km/l na cidade e 12 na estrada não faz 10 em um percurso meio a meio. Há teste fixando esse comportamento.

`buscar_documentos_oficiais` **aceita filtro de tipo**. Ver seção 10.

Seção 7 — Ciclo de vida de uma pergunta

Percurso completo, do clique à resposta:

1. A interface recebe a pergunta e monta o histórico (até 12 turnos anteriores; conversa longa estoura contexto e cota)
2. Gera o identificador da execução
3. `responder()` marca o tempo inicial e invoca o executor
4. O modelo recebe: instruções + histórico + pergunta + esquemas
5. O modelo decide chamar uma ferramenta e emite os argumentos
6. O executor valida os argumentos com `pydantic` e chama a função
7. A função consulta CSV, calcula ou busca no índice, e devolve texto
8. O resultado volta ao modelo, que decide: outra ferramenta ou `responder` (limite de 6 iterações, para não entrar em laço)
9. O modelo redige a resposta final citando as fontes
10. `responder()` extrai o texto puro da saída
11. O registro grava pergunta, resposta, ferramentas, fontes e duração
12. A interface exibe a resposta e os botões de avaliação

O passo 11 acontece em bloco `finally`: a execução é registrada mesmo quando o passo 3 levanta exceção, o que preserva a falha para investigação posterior.

7.1 Limite de iterações

O executor para em **6 iterações**. Sem esse teto, um modelo confuso alterna entre ferramentas indefinidamente, consumindo cota e tempo. Atingir o limite produz a melhor resposta disponível, não erro.

Seção 8 — Pipeline de dados em tempo de construção

8.1 Scripts

Script	Entrada	Saída
<code>coletar_fipe.py</code>	API pública da Tabela FIPE	<code>precos_fipe.csv</code>
<code>coletar_precos_anp.py</code>	CSV aberto da ANP	<code>precos_combustivel_anp.csv</code>
<code>baixar_documentos.py</code>	Portal do Inmetro	PDFs em <code>dados/brutos/documentos/</code>
<code>extrair_consumo_pbev.py</code>	PDFs do Inmetro	<code>consumo_pbev.csv</code>
<code>indexar_documentos.py</code>	Acervo completo	<code>dados/processados/indice_faiss/</code>

8.2 Por que build time

Chamar API de terceiro no caminho crítico de cada pergunta adiciona latência, sujeita o sistema a limite de requisições, torna o resultado não reproduzível e cria ponto de falha externo. Cache, retry e circuit breaker resolveriam — e nada disso se justifica quando os dados mudam mensalmente.

Recalcular embeddings a cada inicialização teria consequência pior: a camada gratuita do provedor limita requisições por dia, e poucas reinicializações esgotariam a cota, deixando o sistema fora do ar.

8.3 Controle de taxa na indexação

A camada gratuita limita requisições **por minuto**, e cada texto do lote conta como uma requisição. A indexação envia lotes de **4 trechos**, com pausa de **4 segundos**, e reenvio com espera crescente ao receber resposta de limite excedido — até 8 tentativas, com espera de 20 a 90 segundos.

O progresso é gravado num índice parcial a cada lote. Uma indexação interrompida — por queda de rede ou esgotamento de cota diária — **retoma de onde parou** na execução seguinte, sem repetir o que já custou crédito.

8.4 Fatiamento

`RecursiveCharacterTextSplitter`, **2400 caracteres com 200 de sobreposição**.

O valor é maior que o usual por causa da cota: um manual de 484 páginas gera 1145 trechos a 1200 caracteres e cerca de 570 a 2400. A recuperação fica ligeiramente menos precisa, e em compensação cada trecho recuperado traz mais contexto ao redor da informação.

Seção 9 — Base de conhecimento e formatos aceitos

9.1 Acervos

Acervo	Local	Tipo atribuído	Versionado
Políticas internas da Autoluz	dados/documentos_corporativos/	documento_interno	Sim
Documentos do Inmetro	dados/brutos/documentos/	documento_oficial	Não, baixado por script
Manuais de montadora	dados/brutos/documentos/manuais/	manual	Não, incluído à mão

9.2 Formatos e tratamento

Formato	Tratamento
PDF	Texto por página, com o número da página preservado para citação
Word (.docx)	Parágrafos e tabelas
Excel (.xlsx)	Uma frase por linha, com o cabeçalho repetido
PowerPoint (.pptx)	Texto de cada slide mais as notas do apresentador
HTML	Texto limpo, descartando script, estilo, navegação e rodapé
CSV	Uma frase por linha, com separador detectado automaticamente
JSON	Achatado em <code>caminho.ate.folha: valor</code>
Markdown e texto	Direto

9.3 Por que tabela vira frase

Uma planilha convertida em texto tabular perde o cabeçalho no primeiro corte do fatiador. O trecho recuperado passa a ser uma fileira de números sem legenda, e o modelo atribui o valor à linha errada — produzindo resposta confiante e falsa.

Repetindo o cabeçalho em cada linha, modelo: Corolla | preco: 145000 sobrevive ao fatiamento e se sustenta isolado. O mesmo raciocínio vale para o JSON: preservar o caminho até a folha mantém `beneficios.vale_refeicao.valor: 44` legível, enquanto um 44 solto não diz nada.

9.4 Metadados por trecho

Cada trecho carrega: `titulo` (declarado no manifesto do acervo), `arquivo`, `formato`, `tipo` e, para PDF, `page`. O título é o que aparece na citação da resposta; sem declaração, o nome do arquivo o substitui.

Seção 10 — Camada de recuperação semântica

10.1 Fluxo

```
pergunta -> embedding -> filtro por metadado -> busca por similaridade
          -> corte por relevância -> trechos com fonte -> contexto do modelo
```

10.2 Filtro por metadado

O agente informa o tipo ao chamar a ferramenta, restringindo o universo **antes** da comparação de vetores. Uma pergunta sobre garantia não deve recuperar trecho da tabela do Inmetro apenas porque ele também fala de veículos.

Salvaguarda: se o filtro não devolver nada, a busca é **refeita no acervo inteiro**. Filtro estreito demais não pode virar "não encontrei".

10.3 Limiar de relevância

Busca vetorial sempre devolve alguma coisa: os vizinhos mais próximos de uma pergunta fora do escopo continuam sendo vizinhos. O limiar descarta o que ficar abaixo da régua e, sem nada acima dela, a ferramenta declara que não encontrou.

O limiar vem desligado por padrão, e isso é decisão consciente. Calibrá-lo exige medir relevâncias de perguntas reais; um corte arbitrado descarta trecho bom em silêncio — falha pior que trecho ruim visível, porque não deixa sintoma. O procedimento de calibração está na seção 17.

10.4 Quantidade recuperada

Quatro trechos por busca, configurável. O valor equilibra contexto suficiente para o modelo e consumo de tokens por pergunta.

Seção 11 — Regras de comportamento e recusa

11.1 Regras do prompt

O prompt do sistema fixa nove regras. As principais:

- 1 Nunca inventar dado — preço, consumo e ficha só vêm de ferramenta;
- 2 Usar `resumo_catalogo` para perguntas de cobertura, nunca uma listagem truncada;
- 3 Nunca citar veículo que não tenha aparecido no retorno de uma ferramenta;

- 4 Nunca calcular de cabeça;
- 5 Declarar quando o veículo está fora do catálogo;
- 6 Citar o mês de referência da FIPE e a fonte do consumo;
- 7 Não dar conselho financeiro, não simular financiamento, não projetar revenda;
- 8 Sem resposta nos documentos, indicar a área responsável;
- 9 Nunca repetir nem solicitar dado pessoal de cliente.

11.2 Recusas esperadas

Situação	Comportamento
Veículo fora do catálogo	Declara e oferece similares vindos de ferramenta
Custo de viagem de elétrico	Explica que há km/l equivalente, não kWh
Manual de modelo não indexado	Declara a ausência em vez de estimar
Nome ambíguo (Corolla / Corolla Cross)	Pede desambiguação antes de simular
Conselho financeiro	Explica o limite e oferece o que consegue fazer
Pergunta sem critério ("o melhor carro")	Pede o critério
Assunto sem documento	Indica a área responsável, com e-mail e prazo
Pergunta com dado pessoal	Responde pela regra geral e remete ao sistema interno

A regra 9 e a última linha merecem destaque: a política de privacidade da empresa **proíbe inserir dado pessoal em ferramenta de IA**. O sistema não depende só da disciplina do usuário — a regra está no prompt.

Seção 12 — Configuração

12.1 Variáveis de ambiente

Variável	Padrão	Função
PROVEDOR_LLM	gemini	Provedor ativo: gemini ou nvidia
GOOGLE_API_KEY	—	Credencial do Gemini
NVIDIA_API_KEY	—	Credencial da NVIDIA
MODELO_CHAT	padrão do provedor	Sobrescreve o modelo de chat
MODELO_EMBEDDING	padrão do provedor	Sobrescreve o modelo de embeddings
TEMPERATURA	0.1	Baixa, para reduzir variação entre execuções
TRECHOS_RECUPERADOS	4	Trechos por busca semântica
LIMiar_RELEVANCIA	0	Corte de relevância; zero desliga
REGISTRAR_EXECUCOES	1	Registro de execução
DIR_REGISTROS	registros/	Destino do registro

12.2 Modelos padrão

Provedor	Chat	Embeddings
Gemini	gemini-3.5-flash-lite	models/gemini-embedding-2
NVIDIA	meta/llama-3.3-70b-instruct	nvidia/nv-embedqa-e5-v5

Restrição crítica: o modelo de embeddings usado na consulta tem de ser o mesmo da indexação. Vetores de modelos diferentes não são comparáveis, e o sintoma de violar isso é recuperação silenciosamente ruim, não erro. Trocar o modelo de embeddings **obriga a reconstruir o índice**.

12.3 Credencial

A chave é gravada por `scripts/configurar_chave.py`, que deduz o provedor pelo prefixo, grava o `.env` com permissão restrita e **nunca imprime o valor**. Em servidor, ela vive em `/etc/agente-carros/ambiente`, com modo 0640, fora do diretório da aplicação.

Seção 13 — Implantação

13.1 Ambiente de produção

Item	Escolha
Nuvem	Oracle Cloud Infrastructure, camada Always Free
Instância	VM.Standard.A1.Flex, ARM Ampere A1, 1 OCPU, 6 GB
Sistema	Ubuntu 24.04 LTS, aarch64
Processo	systemd, usuário próprio sem shell
Exposição	nginx como proxy reverso, portas 80 e 443
Aplicação	Streamlit em 127.0.0.1:8501, inacessível de fora

13.2 Serviços OCI utilizados

Compute, Virtual Cloud Network com security list e Block Volume. O requisito do programa é ao menos um serviço; o desenho usa três.

13.3 Fluxo de publicação

```
máquina local      git push origin main
|
servidor            publicar.sh
|                  1. traz o código
|                  2. sincroniza dependências se requirements mudou
|                  3. atualiza unidade systemd e configuração do nginx
|                  4. reinicia o serviço
|                  5. consulta a sonda de saúde
|
+-- sonda falha -> reverte para a revisão anterior e reinicia
```

O servidor nunca recebe arquivo solto: a única fonte é o repositório, e o que está no ar corresponde sempre a um commit publicado.

13.4 Detalhes que costumam falhar

Filtro duplo de rede. A OCI filtra na security list da VCN e no firewall da instância. Abrir só um dos dois produz porta que responde no localhost e não responde de fora.

WebSocket. O Streamlit depende de WebSocket. Sem os cabeçalhos de upgrade no proxy, a página carrega e trava em "Connecting...".

Tempo de resposta. Uma resposta pode levar dezenas de segundos por encadear chamadas de ferramenta. O timeout padrão de 60 segundos do nginx derrubaria a conexão no meio; a configuração usa 3600.

Reivindicação por ociosidade. Instâncias Always Free ociosas por 7 dias são removidas. A correção definitiva é migrar a conta para Pay As You Go, o que encerra a reivindicação e mantém os recursos gratuitos.

O runbook completo está em docs/DEPLOY.md.

Seção 14 — Observabilidade e registro de execução

14.1 O que é gravado

Cada pergunta respondida gera uma linha de JSON com: identificador, momento, pergunta, resposta, duração em milissegundos, ferramentas acionadas, documentos citados, sessão, interface, provedor, modelo e erro, quando houver.

Cada avaliação gera outra linha, referenciando o identificador da execução.

14.2 Formato e local

JSON Lines, um arquivo por dia, em registros/ no desenvolvimento e em /var/log/agente-carros/ no servidor — fora do diretório da aplicação, para sobreviver a um redeploy. A rotação mantém 30 dias comprimidos.

O formato cresce por acréscimo, nunca precisa ser reescrito inteiro e se lê com ferramentas de linha de comando, sem infraestrutura adicional.

14.3 Garantia de não interferência

O registro **falha em silêncio**. Disco cheio, caminho somente leitura ou permissão negada não interrompem a resposta. Observabilidade que derruba a função que observa é defeito, não recurso. Há teste fixando o comportamento.

14.4 Relatório

scripts/relatorio_execucoes.py agrega o registro e responde:

Pergunta de manutenção	Onde aparece
Quantas perguntas, em quantas sessões	Volume
Quanto o agente demora	Mediana, p95 e máximo
Quais ferramentas são realmente usadas	Contagem por ferramenta
Quais documentos sustentam as respostas	Contagem por fonte

Pergunta de manutenção	Onde aparece
O que foi avaliado mal	Percentual e listagem com - - ruins
Quais perguntas não encontraram material	Lista de candidatas a novo documento

A última linha é a mais valiosa: é o retorno direto do uso para a curadoria do acervo.

14.5 Privacidade do registro

Não há identificação de quem perguntou. A sessão é identificador aleatório por aba aberta, que serve apenas para ligar perguntas de uma mesma conversa.

Seção 15 — Segurança e privacidade

15.1 Credenciais

Nunca no repositório, nunca em `user_data` de instância — que fica legível no console e nos metadados. Arquivo com permissão restrita, lido pelo processo e inacessível a usuário comum.

15.2 Superfície exposta

O Streamlit escuta apenas em loopback. Nada além do nginx alcança a porta da aplicação, mesmo de dentro da máquina. O serviço roda com usuário dedicado sem shell, `NoNewPrivileges`, `ProtectSystem`, `PrivateTmp` e teto de memória.

15.3 Desserialização do índice

O carregamento do índice FAISS usa `pickle`, o que exige o sinalizador `allow_dangerous_deserialization`. É seguro **nesta configuração específica**: o índice é gerado pelo próprio projeto e versionado junto com o código. Índice de origem não confiável nunca deve ser carregado.

15.4 Dados pessoais

O sistema **não trata dados pessoais de clientes**. Não há cadastro, autenticação nem integração com sistema de CRM. A política interna proíbe inserir dado pessoal em ferramenta de IA, e o prompt reforça a proibição.

O acervo corporativo é fictício por construção: nomes de pessoas não aparecem, e e-mails, telefones e endereços usam domínio inventado.

Seção 16 — Testes e qualidade

16.1 Cobertura

113 testes, executados em menos de um segundo. A escolha do que testar segue um critério: cobrir onde um erro produziria **resposta errada com aparência de certeza**.

Área	O que é verificado
Simulação de viagem	Cálculo, arredondamento, ida e volta, comparação de combustíveis
Catálogo	Busca, filtros, ordenação, comparação, casos vazios
Preços por estado	Resolução por UF, ausência de dado, ranking
Configuração	Seleção de provedor e de credencial
Recuperação	Filtro por tipo, refazimento sem filtro, limiar, citação de página
Formatos de documento	Extração de Word, Excel, PowerPoint, HTML, CSV, JSON
Acervo corporativo	Integridade, títulos declarados, cobertura de áreas
Registro de execução	Campos gravados, corte de texto, falha silenciosa

16.2 Testes que fixam decisão

Alguns testes existem para impedir regressão de uma decisão, não para verificar código:

- a soma de litros por trecho, contra a média de consumos;
- o cabeçalho repetido em cada linha de planilha;
- a falha silenciosa do registro;
- o refazimento da busca quando o filtro não devolve nada.

16.3 Verificação de ambiente

`scripts/diagnosticar.py` confere datasets, catálogo, configuração, índice e uma chamada real ao provedor, apontando o comando que resolve cada pendência. O modo `--rapido` pula a rede.

Seção 17 — Manutenção e operação contínua

17.1 Rotinas periódicas

Rotina	Frequência sugerida	Comando
Atualizar preços FIPE	Mensal	<code>python scripts/coletar_fipe.py</code>
Atualizar preços ANP	Mensal	<code>python scripts/coletar_pre cos_anp.py</code>
Atualizar tabelas do Inmetro	Anual	<code>baixar_documentos.py</code> e <code>extrair_consumo_pbev.py</code>
Revisar o acervo corporativo	Semestral	Curadoria pela área responsável
Ler o relatório de execução	Semanal	<code>python scripts/relatorio_e xecucoes.py</code>

17.2 Acrescentar um documento

- 1 Salvar o arquivo em `dados/documentos_corporativos/`, em qualquer formato aceito;
- 2 Declará-lo no `MANIFESTO.md` da pasta, com título — é o que o agente cita como fonte;
- 3 Rodar `python scripts/indexar_documentos.py`;

- 4 Conferir com `python scripts/diagnosticar.py`.

17.3 Acrescentar um veículo

Uma linha em `dados/catalogo_semente.csv`, outra em `fichas_tecnicas.csv` e outra em `dados/mapa_pbev.csv`, sempre com o mesmo identificador, seguidas dos scripts de coleta.

17.4 Calibrar o limiar de relevância

- 1 Manter `LIMIAR_RELEVANCIA=0` e usar o agente normalmente por alguns dias;
- 2 Levantar as perguntas que produziram resposta ruim no relatório, com `--ruins`;
- 3 Identificar a relevância dos trechos recuperados nesses casos;
- 4 Definir o limiar **abaixo** da menor relevância entre os acertos observados;
- 5 Acompanhar a taxa de "não encontrei" após a mudança — subida acentuada indica corte alto demais.

17.5 Curadoria por área

Cada documento tem área responsável, declarada no manifesto. Cabe a ela revisar se o conteúdo indexado ainda é a versão vigente. Documento desatualizado produz resposta errada com fonte citada — pior que ausência de resposta, porque carrega aparência de autoridade.

Seção 18 — Limitações conhecidas

Limitação	Efeito	Caminho de superação
Catálogo fechado em 28 modelos de 2024	Veículos fora não são respondidos	Ampliar a semente e recoletar
Ficha técnica em conferência	Motor, potência e torque não conferidos contra a montadora	Auditoria contra material oficial
Elétricos sem custo de viagem	Há km/l equivalente, não kWh	Incluir kWh/100km por modelo
Apenas o Corolla tem manual indexado	Perguntas de manutenção só funcionam para ele	Indexar os 24 manuais guardados
Preço de combustível por estado	Variação intraestadual não capturada	Apuração por município
Consumo de ensaio	Divergência do consumo real	Inerente à fonte; o agente avisa
Sem reranqueamento	Precisão de recuperação abaixo do possível	Acrescentar modelo de reordenação
Sem autenticação	Todo conteúdo é de circulação irrestrita	Só necessário se entrar documento restrito

Seção 19 — Registros de decisão de arquitetura

ADR-01 — Desenho híbrido em vez de RAG puro. *Decisão:* dados estruturados via consulta com pandas; busca semântica apenas para texto corrido. *Motivo:* similaridade vetorial não ordena por preço nem calcula custo. Um índice único produziria respostas confiantes e erradas em toda pergunta numérica. *Consequência:* mais código de ferramenta, e a fronteira entre os dois mecanismos precisa ficar clara na descrição de cada uma.

ADR-02 — Ingestão versionada em vez de chamada em tempo de execução. *Decisão:* coletar por script e versionar o resultado. *Motivo:* latência, limite de requisições, reprodutibilidade e ponto de falha externo. *Consequência:* os números envelhecem entre coletas, e cada dataset registra a data de referência.

ADR-03 — Embeddings em tempo de construção. *Decisão:* construir o índice offline e versioná-lo. *Motivo:* a camada gratuita limita requisições por dia; recalculá-lo na inicialização esgotaria a cota. *Consequência:* acrescentar documento exige reindexar; o índice ocupa espaço no repositório.

ADR-04 — Arquitetura hexagonal com Protocol. *Decisão:* contratos como Protocol, sem herança. *Motivo:* trocar provedor, base vetorial ou fonte de dados deve custar um arquivo. *Consequência:* uma camada de indireção a mais. O bloqueio da conta NVIDIA comprovou o retorno: a migração custou um arquivo e uma linha.

ADR-05 — Trechos de 2400 caracteres. *Decisão:* fatiar em 2400 com 200 de sobreposição, acima do usual. *Motivo:* reduzir o número de requisições de embedding, limitadas por dia. *Consequência:* recuperação ligeiramente menos precisa, com mais contexto por trecho.

ADR-06 — Limiar de relevância desligado por padrão. *Decisão:* implementar o corte, mas entregá-lo inativo. *Motivo:* limiar não calibrado descarta trecho bom em silêncio. *Consequência:* exige calibração deliberada, com procedimento documentado.

ADR-07 — systemd em vez de contêiner. *Decisão:* processo supervisionado por systemd, sem Docker. *Motivo:* uma aplicação, uma máquina. Contêiner acrescentaria build, registro e camada de rede sem resolver problema existente. *Consequência:* a decisão se inverte com mais de um serviço ou necessidade de replicar o ambiente.

Seção 20 — Glossário

Agente. Programa que recebe uma pergunta, decide quais ferramentas usar, executa-as e redige a resposta.

Chunk (trecho). Pedaco de documento resultante do fatiamento, unidade da busca semântica.

Embedding. Vetor numérico que representa o significado de um texto. Textos de sentido próximo geram vetores próximos.

FAISS. Biblioteca de busca por similaridade em vetores, usada aqui como índice local em disco.

Ferramenta (tool). Função que o modelo pode invocar, com esquema de argumentos declarado.

JSON Lines. Formato em que cada linha do arquivo é um objeto JSON completo.

Limiar de relevância. Nota mínima para um trecho recuperado ser aproveitado.

PBE Veicular. Programa Brasileiro de Etiquetagem Veicular, do Inmetro, que publica consumo e eficiência energética.

Porta e adaptador. Contrato e sua implementação concreta.

RAG. *Retrieval-Augmented Generation* — recuperar trechos pertinentes e entregá-los ao modelo como contexto, em vez de confiar na memória dele.

Tabela FIPE. Referência de preço médio de veículos usados no mercado brasileiro.

Tool calling. Capacidade do modelo de emitir uma chamada de função estruturada em vez de texto livre.

Seção 21 — Processo de atualização deste documento

Este manual acompanha o código no repositório e é atualizado junto com mudanças que alterem comportamento observável: nova ferramenta, troca de provedor, alteração de fatiamento, mudança de ambiente de execução ou nova regra de recusa.

Correção de texto não altera a versão. Mudança de conteúdo técnico incrementa a versão menor; reescrita estrutural incrementa a maior.

Versão	Data	Alteração
1.0.0	agosto de 2026	Emissão inicial

Documentos relacionados: README.md (execução do projeto), docs/DEPLOY.md (runbook de implantação), docs/FONTES.md (procedência dos dados), docs/MANUAIS.md (inclusão de manuais de montadora) e o MANIFESTO.md do acervo corporativo.

Documento interno da Autoluz Veículos, empresa fictícia criada para o Challenge Alura — ONE IA for Tech. Os dados de veículos são reais e de fonte oficial; a empresa e suas políticas são fictícias.